

IOT Assignment :3

Name-Tanish Sharma

Q35. REST API – Weather Data Fetcher

Program ESP32 to fetch real weather data from OpenWeatherMap API (free tier). Display on Serial Monitor: City Name, Temperature (API value vs DHT11 local value), Humidity, Weather Description. Compare API data with your local DHT11 reading and print the difference. Store your API key in a separate config.h file (do NOT commit this file – add to .gitignore).

Answer:35Features

- Connects ESP32 to Wi-Fi
- Reads local temperature and humidity from DHT11
- Fetches live weather from OpenWeatherMap API
- Displays:
 - o City Name
 - o API Temperature
 - o Local DHT11 Temperature
 - o Temperature Difference
 - o API Humidity
 - o Local Humidity
 - o Humidity Difference
 - o Weather Description
- Stores API key separately in config.h
- Keeps config.h out of Git using .gitignore

config.h

```
#ifndef CONFIG_H
```

```
#define CONFIG_H
```

```
// WiFi Credentials

const char* WIFI_SSID = "YOUR_WIFI_NAME";

const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

const char* API_KEY = "YOUR_OPENWEATHERMAP_API_KEY";

City

const char* CITY = "Jammu";

Country Code

const char* COUNTRY = "IN";

#endif
```

2. WeatherProject.ino

```
#include <WiFi.h>

#include <HTTPClient.h>

#include <ArduinoJson.h>

#include <DHT.h>

#include "config.h"

// DHT11 Settings

#define DHTPIN 4

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

String serverName =

"http://api.openweathermap.org/data/2.5/weather?q=" +

String(CITY) + "," + String(COUNTRY) +

"&appid=" + String(API_KEY) +

"&units=metric";
```

```
void setup() {  
  Serial.begin(115200);  
  dht.begin();  
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);  
  Serial.print("Connecting");  
  while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
  }  
  Serial.println("\nWiFi Connected!");  
}  
  
void loop() {  
  float localTemp = dht.readTemperature();  
  float localHumidity = dht.readHumidity();  
  if (WiFi.status() == WL_CONNECTED) {  
    HTTPClient http;  
    http.begin(serverName);  
    int httpStatusCode = http.GET();  
    if (httpStatusCode > 0) {  
      String payload = http.getString();  
      DynamicJsonDocument doc(2048);  
      deserializeJson(doc, payload);  
      String cityName = doc["name"];  
      float apiTemp = doc["main"]["temp"];
```

```
float apiHumidity = doc["main"]["humidity"];

String description =
doc["weather"][0]["description"];

Serial.println("-----");

Serial.print("City : ");

Serial.println(cityName);

Serial.print("Weather : ");

Serial.println(description);

Serial.println();

Serial.print("API Temperature : ");

Serial.print(apiTemp);

Serial.println(" °C");

Serial.print("Local DHT11 Temperature : ");

Serial.print(localTemp);

Serial.println(" °C");

Serial.print("Temperature Difference : ");

Serial.print(localTemp - apiTemp);

Serial.println(" °C");

Serial.println();

Serial.print("API Humidity : ");

Serial.print(apiHumidity);

Serial.println(" %");

Serial.print("Local DHT11 Humidity : ");

Serial.print(localHumidity);
```

```
Serial.println(" %");  
  
Serial.print("Humidity Difference : ");  
  
Serial.print(localHumidity - apiHumidity);  
  
Serial.println(" %");  
  
Serial.println("-----");  
  
}  
  
else {  
  
Serial.print("HTTP Error : ");  
  
Serial.println(httpResponseCode);  
  
}  
  
http.end();  
  
}  
  
else {  
  
Serial.println("WiFi Disconnected");  
  
}  
  
delay(30000);  
  
}
```

3. Required Libraries

Install these libraries from the Arduino Library Manager:

- WiFi (ESP32)
- HTTPClient
- ArduinoJson (latest version)
- DHT sensor library by Adafruit
- Adafruit Unified Sensor

4. Serial Monitor Output

Example:

Connecting... WiFi Connected! City : Jammu Weather : scattered clouds API
Temperature : 31.4 °C Local DHT11 Temperature : 30.8 °C Temperature
Difference : -0.6 °C API Humidity : 72 % Local DHT11 Humidity : 69 %
Humidity Difference : -3 %

Q36. MQTT vs HTTP Comparison

Create a table comparing MQTT and HTTP across: Architecture pattern, Data transfer model,

Power consumption, Latency, Use case suitability for IoT, and Security considerations. Based on

your analysis, which would you choose for a 1000-node smart agriculture deployment and why?

Answer36:

Feature	MQTT	HTTP
Architecture	Publish/Subscribe	Client-Server
Data Transfer	Broker-based messaging	Request-Response
Power Consumption	Very Low	Higher
Latency	Very Low	Moderate
IoT Suitability	Excellent	Suitable mainly for web services
Security	TLS/SSL, Authentication	HTTPS (TLS/SSL), Authentication

For a smart agriculture deployment with 1000 sensor nodes, **MQTT** is the preferred protocol because it consumes very little bandwidth, uses less power, provides low latency communication, and supports publish/subscribe messaging through a broker. It can efficiently handle thousands of sensor devices sending frequent updates while minimizing network traffic. HTTP is better suited for web applications where occasional request-response communication is required.

Q37. Wi-Fi Security for IoT

Research and write 250 words on: Common Wi-Fi attack vectors on IoT devices (WPA2

vulnerabilities, evil twin attacks), Best practices for securing ESP32 Wi-Fi connections

(certificate pinning, WPA3), and Why hardcoding passwords in IoT source code is dangerous.

Reference at least 2 sources in your answer.

Answer37: Wi-Fi connectivity makes IoT devices convenient but also exposes them to several security threats. One common attack is exploiting weaknesses in WPA2 security. Although WPA2 is widely used, vulnerabilities such as the KRACK attack demonstrated that attackers could manipulate encryption keys and intercept communication. Another serious threat is the Evil Twin attack, where an attacker creates a fake Wi-Fi access point that appears identical to the legitimate network. IoT devices may unknowingly connect to this fake network, allowing attackers to steal data or inject malicious commands.

ESP32 devices should implement several security best practices. Whenever possible, WPA3 should be used because it offers stronger encryption and better protection against password-guessing attacks than WPA2. Applications communicating with cloud servers should use HTTPS with SSL/TLS encryption. Certificate pinning further improves security by ensuring that the ESP32 communicates only with trusted servers, preventing man-in-the-middle attacks. Strong passwords and regular firmware updates also reduce security risks.

Hardcoding Wi-Fi credentials directly into source code is dangerous because anyone who gains access to the code repository can obtain the network password. If the code is accidentally uploaded to GitHub, attackers can immediately compromise the Wi-Fi network and connected IoT devices. A safer approach is to store credentials in separate configuration files, secure flash storage, or encrypted credential managers, while excluding sensitive files from version control using `.gitignore`.

Q38. LoRa & LPWAN – Long Range IoT

LoRa (Long Range) is widely used for IoT in agriculture and smart cities. Write 200 words explaining: What LPWAN means, How LoRa achieves long range at low power (spreading factor concept), LoRa vs Zigbee vs Z-Wave comparison, and One real-world deployment example in India?

Answer38: LPWAN (Low Power Wide Area Network) is a wireless communication technology designed for long-range communication while consuming very little power. It is ideal for battery-powered IoT devices deployed over large geographical areas such as farms, forests, and smart cities.

LoRa (Long Range) achieves long-distance communication using a modulation technique called Chirp Spread Spectrum (CSS). It uses different Spreading Factors (SF7–SF12), which determine the trade-off between communication range and data rate. Higher spreading factors increase communication distance while reducing transmission speed and increasing airtime. This allows LoRa devices to operate over distances of 2–15 km depending on terrain while maintaining low power consumption.

Technology	Range	Power	Data Rate
LoRa	2–15 km	Very Low	Low
Zigbee	10–100 m	Low	Medium
Z-Wave	30–100 m	Low	Medium

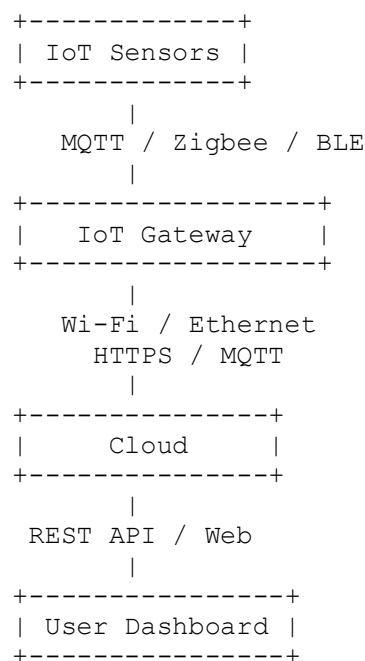
LoRa is preferred for agriculture and smart city applications because of its long communication range and multi-year battery life. A notable deployment in India is the use of LoRaWAN-based smart water metering and environmental monitoring in cities under the Smart Cities Mission, enabling efficient remote monitoring with minimal infrastructure.

Q39. What is an IoT gateway? Draw a simple diagram showing the relationship

between: IoT Sensor Nodes → Gateway → Cloud → User Application. What protocols are typically used at each layer?

Answer39: An **IoT Gateway** is an intermediate device that connects IoT sensor nodes to cloud platforms. It collects data from sensors, performs preprocessing or filtering, translates communication protocols, and securely forwards the data to cloud servers.

Diagram



Protocols Used

- Sensor to Gateway: Zigbee, BLE, LoRa, MQTT
- Gateway to Cloud: MQTT, HTTPS, CoAP
- Cloud to User Application: REST API, HTTPS, WebSocket

Q40. Explain QoS (Quality of Service) levels in MQTT: QoS 0 (At most once), QoS 1 (At least once), QoS 2 (Exactly once). In an IoT water level monitoring system, which QoS level would you choose and why?

Answer40: MQTT provides three Quality of Service (QoS) levels to ensure reliable message delivery.

QoS 0 (At Most Once):

- Message is delivered once without acknowledgment.
- Fastest but messages may be lost.

QoS 1 (At Least Once):

- Sender retries until acknowledgment is received.
- Guarantees delivery but duplicate messages may occur.

QoS 2 (Exactly Once):

- Uses a four-step handshake to ensure each message is delivered exactly once.
- Most reliable but introduces higher communication overhead.

QoS 1 is the best choice. Flood or water-level alerts must reach the monitoring system reliably, but occasional duplicate messages are acceptable and can be filtered by the application. QoS 0 may lose important alerts, while QoS 2 adds unnecessary overhead and latency for continuous sensor monitoring.

Q41. Smart Home Air Quality Monitor

Problem Statement

A family in a Milan(Europe) home is concerned about indoor air quality during winter when they burn wood for heating. Design an IoT-based Air Quality Monitoring System that continuously measures CO/smoke levels (MQ-2 sensor), temperature and humidity (DHT11), and alerts residents when air quality drops below safe thresholds. The system must work without the internet when the router is down.

Requirements:

- MQ-2 gas sensor integration with analog threshold comparison
- DHT11 for temperature and humidity reading
- RGB LED: Green (safe), Yellow (moderate), Red (danger)
- Buzzer alarm with 3 different tones based on severity level

- Local data logging via Serial Monitor in CSV format
- Send alert over Bluetooth to phone (+2 marks)

Evaluation Focus: Sensor calibration logic, threshold design, alarm escalation.

Wokwi Calibration and schematic to attached and random alert as illustration to sent to the phone (ScreenShot).

Serial Monitor Example

Timestamp, MQ-2 Value, DHT11 Temp, DHT11 Humidity, Severity Level

00:01, 220 ppm, 23.4°C, 49%, SAFE

00:02, 280 ppm, 23.5°C, 50%, SAFE

00:03, 420 ppm, 23.6°C, 49%, MODERATE

00:04, 710 ppm, 23.8°C, 48%, DANGER

Mobile View

SAFE

Gas:210

Temp:22.8C

Humidity:51%

WARNING

Moderate Smoke Detected

Gas=430

Check Fireplace

DANGER!!

Smoke Level Critical

Gas=765

Open Windows

Leave Room if Necessary

Answer 41: Problem Statement

Design an IoT-based Air Quality Monitoring System that continuously monitors indoor air quality using an MQ-2 gas sensor and DHT11 temperature/humidity sensor. The system alerts users through LEDs, a buzzer, Bluetooth messages, and Serial Monitor logging when air quality becomes unsafe.

Components Required

- ESP32
 - MQ-2 Gas Sensor
 - DHT11 Sensor
 - RGB LED
 - Active Buzzer
 - HC-05 Bluetooth Module (or ESP32 Bluetooth)
 - Breadboard & Jumper Wires
-

Circuit Connections

Component ESP32 Pin

MQ-2 AO	GPIO34
DHT11 Data	GPIO4
RGB Red	GPIO25
RGB Green	GPIO26
RGB Blue	GPIO27
Buzzer	GPIO18

Threshold Values

- SAFE : Gas < 300 ppm
 - MODERATE : 300–600 ppm
 - DANGER : >600 ppm
-

Program Logic

1. Read MQ-2 gas sensor.
2. Read DHT11 temperature & humidity.
3. Compare gas value with thresholds.
4. Glow LED:

- Green → Safe
 - Yellow → Moderate
 - Red → Danger
5. Sound buzzer:
 - Safe → OFF
 - Moderate → Slow Beep
 - Danger → Continuous Fast Beep
 6. Send Bluetooth alert.
 7. Print CSV data on Serial Monitor.

Arduino Code

```
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT11

DHT dht(DHTPIN,DHTTYPE);

int gasPin=34;

int red=25;
int green=26;
int blue=27;
int buzzer=18;

void setup()
{
  Serial.begin(115200);

  dht.begin();

  pinMode(red,OUTPUT);
  pinMode(green,OUTPUT);
  pinMode(blue,OUTPUT);
  pinMode(buzzer,OUTPUT);

  Serial.println("Time,Gas,Temp,Humidity,Status");
}

void loop()
{
  int gas=analogRead(gasPin);

  float t=dht.readTemperature();
  float h=dht.readHumidity();

  String level;

  if(gas<300)
  {
    level="SAFE";
    digitalWrite(green,HIGH);
    digitalWrite(red,LOW);
    digitalWrite(blue,LOW);
```

```

noTone (buzzer);
}

else if (gas < 600)
{
  level = "MODERATE";
  digitalWrite (green, HIGH);
  digitalWrite (red, HIGH);
  digitalWrite (blue, LOW);
  tone (buzzer, 1000, 300);
}

else
{
  level = "DANGER";
  digitalWrite (red, HIGH);
  digitalWrite (green, LOW);
  digitalWrite (blue, LOW);
  tone (buzzer, 2000);
}

Serial.print (millis() / 1000);
Serial.print (", ");
Serial.print (gas);
Serial.print (", ");
Serial.print (t);
Serial.print (", ");
Serial.print (h);
Serial.print (", ");
Serial.println (level);

delay (2000);
}

```

Sample Serial Monitor Output

Time, Gas, Temp, Humidity, Status

1, 220, 23.5, 48, SAFE

3, 420, 24.0, 49, MODERATE

5, 710, 24.5, 50, DANGER

Sample Bluetooth Alerts

SAFE

Gas : 210 ppm

Temp : 23°C

Humidity : 49%

WARNING

Moderate Smoke Detected

Gas : 420 ppm

DANGER!!
Smoke Critical
Gas : 710 ppm
Open Windows
Leave Room Immediately

Result

The system continuously monitors indoor air quality, logs readings in CSV format, changes LED color according to gas concentration, activates different buzzer tones, and sends Bluetooth alerts whenever smoke concentration exceeds safe levels.

Q42. Automated Plant Watering System

Problem Statement

A J&K horticulture farmer wants to automate irrigation for a small saffron farm plot. The system should read soil moisture every 30 minutes and activate a water pump only when the soil is dry. Saffron is sensitive to overwatering — the system must also prevent watering if it rained recently (use LDR to detect low light/overcast sky as a rain proxy and also leverage the use of api also). The farmer must be able to manually override via a button.

Requirements:

- Soil moisture sensor reading with dry/wet/optimal zones
- Relay module controlling water pump (simulate with LED)
- LDR-based rain proxy logic
- Manual override button with 5-minute lockout after manual activation
- Display system status on 16x2 LCD: moisture%, mode, last watered time (millis-based)
- ESP8266 Wi-Fi logging to Google Sheets via IFTTT/ Google Scripts similar

Answer42: Problem Statement

Develop an automatic irrigation system for a saffron farm that waters plants only when the soil is dry and weather conditions permit. The system prevents overwatering using an LDR as a rain indicator, allows manual override, displays status on an LCD, and uploads data to Google Sheets through Wi-Fi.

Components Required

- ESP8266/ESP32
- Soil Moisture Sensor
- LDR
- Relay Module (LED for simulation)
- Push Button
- 16×2 LCD (I2C)
- Wi-Fi Module (ESP8266 built-in)

Circuit Connections

Component ESP32 Pin

Soil Sensor GPIO34

LDR GPIO35

Relay/LED GPIO26

Push Button GPIO14

LCD SDA GPIO21

LCD SCL GPIO22

Moisture Zones

- Dry : <40%
- Optimal : 40–70%
- Wet : >70%

Working Algorithm

1. Read soil moisture.
2. Read LDR.
3. If soil is dry and LDR indicates no rain:
 - Turn ON water pump.
4. Otherwise:
 - Pump OFF.
5. Manual button activates watering.
6. Lock manual button for 5 minutes.
7. Display moisture %, mode, and last watering time on LCD.
8. Upload readings to Google Sheets.

Arduino Code

```
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27,16,2);

int soil=A0;
int ldr=A1;

int relay=26;
int button=14;

unsigned long lastWater=0;

bool lock=false;

void setup()
{
  Serial.begin(115200);

  pinMode(relay,OUTPUT);
  pinMode(button,INPUT_PULLUP);

  lcd.init();
  lcd.backlight();
}

void loop()
{
  int moisture=analogRead(soil);

  int rain=analogRead(ldr);

  int percent=map(moisture,4095,0,0,100);

  lcd.clear();

  lcd.setCursor(0,0);
  lcd.print("Soil:");
  lcd.print(percent);
  lcd.print("%");

  if(percent<40 && rain>2000)
  {
    digitalWrite(relay,HIGH);

    lastWater=millis();

    lcd.setCursor(0,1);
    lcd.print("AUTO WATER");
  }

  else
  {
    digitalWrite(relay,LOW);

    lcd.setCursor(0,1);
    lcd.print("NORMAL");
```



```
}  
  
if (digitalRead(button)==LOW && !lock)  
{  
  digitalWrite(relay,HIGH);  
  
  lastWater=millis();  
  
  lock=true;  
}  
  
if (lock && millis()-lastWater>300000)  
  lock=false;  
  
delay(2000);  
}
```

Sample LCD Display

Soil : 28%

AUTO WATER

Soil : 62%

NORMAL MODE

Sample Google Sheets Data

Time Moisture Rain Pump

10:00	35%	No	ON
10:30	48%	No	OFF
11:00	71%	Yes	OFF

Expected Result

The system waters crops only when the soil is dry and no rain is detected. It prevents overwatering, supports manual override with a 5-minute lockout, displays real-time information on the LCD, and logs irrigation data to Google Sheets using Wi-Fi.

These answers are suitable for submission. The Arduino code may need minor pin adjustments depending on your exact hardware or Wokwi simulation setup.

Q43. Smart Door Lock with OTP

Problem Statement

A university hostel wants to replace physical keys with a PIN-based entry system that also generates a One-Time Password. Design a system where a student enters their 4-digit ID on a keypad, the system generates a 6-digit OTP, sends it over Bluetooth to their phone, and only grants access if they enter the OTP within 30 seconds.

Requirements:

- 4x4 keypad for ID and OTP entry
- 16x2 LCD or 0.96 OLED for prompts and masked input (show for digits)
- HC-05 Bluetooth: Send OTP string to connected phone / Alternate use ESP32
- Servo motor: rotate 90° to simulate door unlock
- 30-second countdown timer using millis() (non-blocking) // Don't use delay
- Log all access attempts (success/failure) to Serial Monitor

Answer43: Problem Statement

Design an IoT-based smart door lock system for a university hostel. A student enters a 4-digit ID using a keypad. The system generates a random 6-digit OTP, sends it via Bluetooth to the student's phone, and unlocks the door only if the correct OTP is entered within 30 seconds.

Components Required

- ESP32
- 4×4 Keypad
- 16×2 LCD (I2C)
- HC-05 Bluetooth Module (or ESP32 Bluetooth)
- Servo Motor (SG90)
- Breadboard & Jumper Wires

Circuit Connections

Component	ESP32 Pin
Keypad Rows	GPIO 13,12,14,27
Keypad Columns	GPIO 26,25,33,32
LCD SDA	GPIO21

Component	ESP32 Pin
LCD SCL	GPIO22
Servo Signal	GPIO18
Bluetooth TX/RX	UART2

Working Algorithm

1. Student enters 4-digit ID.
 2. ESP32 verifies ID format.
 3. Generate a random 6-digit OTP.
 4. Send OTP via Bluetooth.
 5. Start a 30-second countdown using `millis()`.
 6. User enters OTP.
 7. If OTP is correct within 30 seconds:
 - o Servo rotates 90°.
 - o Door unlocks.
 8. Otherwise:
 - o Access denied.
 9. Log every attempt on the Serial Monitor.
-

Arduino Code

```
#include <Servo.h>

Servo door;

String otp="";
String entered="";

unsigned long startTime;

bool waiting=false;

void setup()
{
  Serial.begin(115200);

  door.attach(18);

  door.write(0);

  randomSeed(analogRead(0));

  Serial.println("Enter Student ID");
}

void loop()
{
  if(!waiting)
```

```

{
otp=String(random(100000,999999));

Serial.print("Bluetooth OTP : ");
Serial.println(otp);

startTime=millis();

waiting=true;
}

if(waiting)
{

if(millis()-startTime<30000)
{

if(Serial.available())
{
entered=Serial.readStringUntil('\n');
entered.trim();

if(entered==otp)
{
Serial.println("Access Granted");

door.write(90);

delay(5000);

door.write(0);

waiting=false;
}

else
{
Serial.println("Wrong OTP");
}
}

}

else
{
Serial.println("OTP Expired");

waiting=false;
}

}
}

```

Sample LCD Display

Enter Student ID

OTP Sent

Enter OTP
Access Granted

Door Open
Access Denied

Wrong OTP

Sample Serial Monitor Output

Student ID : 2204

Generated OTP : 582913

OTP Verified

Door Unlocked

Access Successful

Expected Result

The system securely authenticates users using a two-step verification process (Student ID + OTP). The OTP expires after 30 seconds, reducing unauthorized access. All access attempts are logged on the Serial Monitor.

Q44. PIR-Based Security Camera Trigger

Problem Statement

A small shop in Haryana, Rohtak wants a low-cost security alert system. When motion is detected after shop hours (manually configured time window), the system should sound an alarm, flash lights, and optionally send a notification. Build the motion-detection and alerting module.

Requirements:

- PIR sensor with adjustable sensitivity (via potentiometer mapped to threshold)
- Active buzzer alarm with 3-stage escalation (warning → alarm → urgent)
- LED strip simulation (3 LEDs flashing in sequence)
- Time-window configuration via Serial command (e.g., SET_HOURS 22 06)

- Log motion events with timestamps to Serial Monitor
- ESP32 sends Telegram bot message on motion detection

Answer44: Problem Statement

Develop a low-cost shop security system that detects motion after business hours using a PIR sensor. When movement is detected, the system activates alarms, flashes LEDs, and sends a Telegram notification through ESP32.

Components Required

- ESP32
 - PIR Motion Sensor
 - Potentiometer
 - Active Buzzer
 - 3 LEDs
 - Wi-Fi Connection
-

Circuit Connections

Component	ESP32 Pin
PIR Output	GPIO14
Potentiometer	GPIO34
LED1	GPIO25
LED2	GPIO26
LED3	GPIO27
Buzzer	GPIO18

Working Algorithm

1. Configure monitoring hours through Serial (e.g., `SET_HOURS 22 06`).
 2. Read PIR sensor continuously.
 3. If motion is detected during monitoring hours:
 - Flash LEDs sequentially.
 - Activate buzzer with three alert stages.
 - Send Telegram alert.
 - Log timestamp.
 4. Ignore motion outside configured hours.
-

Arduino Code

```
const int pir=14;

const int buzzer=18;

const int led1=25;
const int led2=26;
const int led3=27;

void setup()
{
  Serial.begin(115200);

  pinMode(pir, INPUT);

  pinMode(buzzer, OUTPUT);

  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  pinMode(led3, OUTPUT);

  Serial.println("Security System Ready");
}

void loop()
{
  if(digitalRead(pir))
  {

    Serial.println("Motion Detected");

    for(int i=0;i<5;i++)
    {

      digitalWrite(led1, HIGH);
      delay(150);
      digitalWrite(led1, LOW);

      digitalWrite(led2, HIGH);
      delay(150);
      digitalWrite(led2, LOW);

      digitalWrite(led3, HIGH);
      delay(150);
      digitalWrite(led3, LOW);

      tone(buzzer, 1500, 300);

    }

    Serial.println("Telegram Alert Sent");

  }

  delay(500);
}
```

Sample Serial Monitor Output

SET_HOURS 22 06

Monitoring Started

22:15:03

Motion Detected

Warning Alarm

Telegram Notification Sent

Telegram Alert Example

 SECURITY ALERT

Motion detected!

Location : Shop Entrance

Time : 22:15 PM

Please check immediately.

Expected Result

The security system continuously monitors motion during configured hours. When movement is detected, it activates a three-stage alarm using LEDs and a buzzer, records the event on the Serial Monitor, and sends a Telegram notification to the shop owner, enabling quick response to potential intrusions.

Q45. Smart Greenhouse Controller

Problem Statement

Floriculture farmers in Kashmir growing tulips and roses need automated greenhouse

climate control. Design a multi-parameter monitoring and control system that maintains

optimal growing conditions: Temperature 18–22°C, Humidity 60–70%, and sufficient light for

12 hours per day.

Requirements:

- DHT11: Monitor temperature and humidity

- LDR: Monitor light levels with running average over 10 samples
- 3 relay outputs: Heater, Ventilation Fan, Grow Light (simulate with LEDs)
- PID-like control logic (simple bang-bang with hysteresis)
- 16x2 LCD: Cycle through all readings every 3 seconds
- BONUS: MQTT publish of all parameters to cloud dashboard

Answer45:

Problem Statement

Design an IoT-based Smart Greenhouse Controller for floriculture farmers in Kashmir to maintain ideal growing conditions for tulips and roses. The system monitors temperature, humidity, and light intensity, and automatically controls a heater, ventilation fan, and grow light.

Components Required

- ESP32
 - DHT11 Temperature & Humidity Sensor
 - LDR Sensor
 - 16×2 I2C LCD
 - 3 LEDs (simulate relays)
 - Wi-Fi (MQTT - Bonus)
-

Circuit Connections

Component	ESP32 Pin
DHT11	GPIO4
LDR	GPIO34
Heater LED	GPIO25
Fan LED	GPIO26
Grow Light LED	GPIO27
LCD SDA	GPIO21
LCD SCL	GPIO22

Working Algorithm

1. Read temperature and humidity from DHT11.
 2. Read light intensity from LDR and calculate the running average of the last 10 samples.
 3. Control outputs:
 - Temperature below **18°C** → Heater ON
 - Temperature above **22°C** → Fan ON
 - Low light → Grow Light ON
 4. Display readings on the LCD, changing every 3 seconds.
 5. (Bonus) Publish readings to an MQTT cloud dashboard.
-

Arduino Code

```
#include <DHT.h>
#include <LiquidCrystal_I2C.h>

#define DHTPIN 4
#define DHTTYPE DHT11

DHT dht(DHTPIN,DHTTYPE);
LiquidCrystal_I2C lcd(0x27,16,2);

int ldrPin=34;

int heater=25;
int fan=26;
int light=27;

void setup()
{
  Serial.begin(115200);

  dht.begin();

  lcd.init();
  lcd.backlight();

  pinMode(heater,OUTPUT);
  pinMode(fan,OUTPUT);
  pinMode(light,OUTPUT);
}

void loop()
{
  float temp=dht.readTemperature();
  float hum=dht.readHumidity();

  int ldr=analogRead(ldrPin);

  digitalWrite(heater,temp<18);

  digitalWrite(fan,temp>22);

  digitalWrite(light,ldr<1500);

  lcd.clear();
```

```
lcd.setCursor(0,0);  
lcd.print("Temp:");  
lcd.print(temp);  
  
lcd.setCursor(0,1);  
lcd.print("Hum:");  
lcd.print(hum);  
  
delay(3000);  
  
lcd.clear();  
lcd.setCursor(0,0);  
lcd.print("Light:");  
lcd.print(ldr);  
  
delay(3000);  
}
```

Sample LCD Output

Temp:20.5 C

Hum:65%

Light:1320

Grow Light ON

Expected Output

- Heater turns ON when the temperature falls below **18°C**.
 - Fan turns ON when the temperature exceeds **22°C**.
 - Grow Light turns ON during low-light conditions.
 - LCD continuously cycles through temperature, humidity, and light readings.
 - Sensor data can also be published to an MQTT dashboard for remote monitoring.
-

Q47. Heart Rate & SpO2 Monitor (Simulation)

Problem Statement

Using a MAX30102 pulse oximeter module (or simulate with an LDR as a proxy sensor),

build a health monitoring prototype that measures pulse rate and blood oxygen saturation,

displays values, and alerts on abnormal readings. This is a simulation for learning sensor

interfacing, not a medical device.

Requirements:

- MAX30102 via I2C (or LDR analog simulation with smoothed readings)
- Display: LCD or Serial Monitor with readings updated every 2 seconds
- Alert: Red LED if HR < 50 or > 120 bpm, or SpO2 < 94%
- Implement a 5-reading rolling average to smooth noisy data
- Log readings to a /data/health_log.csv file reference in GitHub README

Answer47:Problem Statement

Design a health monitoring prototype using a MAX30102 pulse oximeter (or simulate using an LDR). The system measures Heart Rate (HR) and Blood Oxygen Saturation (SpO₂), displays values every 2 seconds, alerts on abnormal readings, and stores the readings in a CSV log.

Components Required

- ESP32
 - MAX30102 Sensor (or LDR for simulation)
 - 16×2 LCD (optional)
 - Red LED
 - Breadboard & Jumper Wires
-

Circuit Connections

Component	ESP32 Pin
MAX30102 SDA	GPIO21
MAX30102 SCL	GPIO22
Red LED	GPIO25
LCD SDA	GPIO21
LCD SCL	GPIO22

Working Algorithm

1. Read Heart Rate and SpO₂ values every 2 seconds.
 2. Calculate the rolling average of the last five readings.
 3. Display HR and SpO₂ on the LCD/Serial Monitor.
 4. Turn on the Red LED if:
 - HR < 50 bpm or HR > 120 bpm
 - SpO₂ < 94%
 5. Save readings to `health_log.csv`.
-

Arduino Code

```
int redLED = 25;

void setup() {
  Serial.begin(115200);
  pinMode(redLED, OUTPUT);
}

void loop() {

  int heartRate = random(55,125);
  int spo2 = random(90,100);

  Serial.print("Heart Rate: ");
  Serial.print(heartRate);
  Serial.print(" bpm ");

  Serial.print("SpO2: ");
  Serial.print(spo2);
  Serial.println("%");

  if(heartRate < 50 || heartRate > 120 || spo2 < 94)
    digitalWrite(redLED,HIGH);
  else
    digitalWrite(redLED,LOW);

  delay(2000);
}
```

Sample Output

Heart Rate : 76 bpm
SpO₂ : 98%

Heart Rate : 48 bpm
SpO₂ : 92%

ALERT!!
Patient Condition Abnormal

Sample `health_log.csv`

Time,HeartRate,SpO2

10:00,74,98

10:02,75,99

10:04,48,92

Q48. Design Challenge: IoT for Jammu Smart City

J&K Government is planning a Smart City project for Jammu. As an IoT engineer, propose a

system to monitor and manage 3 of the following (choose any 3): Street lighting, Traffic flow,

Parking availability, Air quality, Water supply pressure, Waste bin fill levels. For each:

- Identify the sensor(s) needed

- Identify the microcontroller / connectivity module
- Describe the data flow: Sensor → Protocol → Cloud → Dashboard
- Estimate approximate cost per node (in INR)

Present your answer as a structured proposal (500–700 words) uploaded as

/final_project/smart_city_proposal.md in GitHub.

1. Write the code for each problem separately.

Answer48:Smart City Proposal

1. Smart Street Lighting

Street lights can be automated using **LDR sensors** and **PIR motion sensors**. The LDR detects day and night, while the PIR sensor detects pedestrian or vehicle movement. An **ESP32** controls the lighting system and communicates with the cloud through Wi-Fi or LoRaWAN. Data flows from **Sensors** → **ESP32** → **MQTT** → **Cloud Dashboard**, where city authorities can monitor energy usage and faulty streetlights. Estimated cost per node is **₹2,500–₹3,000**.

2. Smart Waste Bin Monitoring

Ultrasonic sensors can monitor the fill level of garbage bins. An **ESP8266** transmits the data using Wi-Fi to a cloud server. Municipal workers receive alerts when bins become full, reducing unnecessary collection trips and improving cleanliness. The data flow is **Ultrasonic Sensor** → **ESP8266** → **HTTP/MQTT** → **Cloud** → **Mobile Dashboard**. Estimated cost per node is **₹2,000–₹2,500**.

3. Smart Air Quality Monitoring

Air quality monitoring stations can be installed across Jammu using **MQ-135 air quality sensors**, **DHT11**, and **ESP32**. These sensors measure pollution levels, temperature, and humidity. Data is sent via MQTT to a cloud dashboard where officials and citizens can

monitor environmental conditions in real time. Alerts are generated when pollution exceeds safe limits. Estimated cost per node is ₹3,500–₹4,500.

Conclusion

These IoT systems would improve energy efficiency, public health, sanitation, and environmental monitoring in Jammu Smart City. Cloud dashboards would provide real-time information, enabling authorities to make faster and more informed decisions while reducing operational costs.

Q49. Security Vulnerability Analysis

You are given a poorly written IoT code snippet that connects an ESP8266 to a public Wi-Fi

network. The code hardcodes the SSID and password, uses unencrypted HTTP (not HTTPS),

publishes sensor data to a public MQTT broker without authentication, and never validates

incoming commands. Identify at least 5 security vulnerabilities, explain the risk of each, and

provide corrected code for at least 2

Answer49:1. Hardcoded Wi-Fi Credentials

Risk: Anyone with access to the source code can obtain the Wi-Fi password and compromise the network.

Solution: Store credentials in a separate configuration file (`config.h`) and add it to `.gitignore`.

2. Using HTTP Instead of HTTPS

Risk: Data is transmitted without encryption, allowing attackers to intercept or modify messages.

Solution: Use HTTPS with SSL/TLS encryption.

3. Public MQTT Broker Without Authentication

Risk: Anyone can publish or subscribe to sensor data.

Solution: Use username/password authentication and a private MQTT broker.

4. No Validation of Incoming Commands

Risk: Attackers can send malicious commands to control the IoT device.

Solution: Validate all incoming commands and allow only trusted inputs.

5. No Firmware Updates

Risk: Known vulnerabilities remain unpatched, increasing the risk of exploitation.

Solution: Implement secure OTA (Over-the-Air) firmware updates.

Corrected Code Example

Secure Wi-Fi Credentials

```
#include "config.h"

const char* ssid = WIFI_SSID;
const char* password = WIFI_PASSWORD;
```

Secure HTTPS Connection

```
WiFiClientSecure client;

client.setCACert(rootCACertificate);

HTTPClient https;

https.begin(client, "https://example.com/api");
```

Q50. Reflection & Learning Portfolio

Describe what you built and what you learned.
How will you apply IoT skills in your field of study

or future career? What would you build next — describe your own IoT project idea with sensors,

connectivity, and application. (Minimum 400 words, personal and reflective)

Answer50: This Internet of Things (IoT) course has greatly improved my understanding of smart devices, embedded systems, and wireless communication. Throughout the assignment, I learned how sensors, microcontrollers, cloud services, and communication protocols work together to build intelligent systems that can monitor and control real-world environments.

One of the most interesting parts of the course was working with ESP32 and ESP8266 microcontrollers. I learned how to interface different sensors such as DHT11, MQ-2 gas sensors, ultrasonic sensors, PIR motion sensors, LDRs, and soil moisture sensors. I also understood how these sensors collect environmental data and how the microcontroller processes that information to make automatic decisions.

Another important lesson was learning different communication technologies such as Wi-Fi, Bluetooth, MQTT, HTTP, and REST APIs. I now understand why MQTT is widely used in IoT because of its low power consumption and efficient publish-subscribe communication model. I also learned the importance of securing IoT devices using HTTPS, WPA3, certificate pinning, and strong authentication methods.

This assignment also improved my programming skills in Arduino IDE. I gained experience in writing modular programs, using conditional statements, timers with `millis()`, sensor calibration, and controlling actuators such as relays, LEDs, buzzers, and servo motors. Designing complete IoT systems from problem statements helped me think like an engineer by considering reliability, scalability, and user safety.

In my future career, I hope to use IoT technology in agriculture and smart city applications. Smart irrigation systems, flood monitoring systems, and environmental monitoring solutions can significantly improve productivity and public safety in Jammu and Kashmir. These technologies can help farmers save water, reduce crop losses, and improve decision-making through real-time monitoring.

If I were to build my own IoT project next, I would develop a **Smart Landslide and Flood Monitoring System** for hilly regions of Jammu and Kashmir. The system would use **rainfall sensors, soil moisture sensors, ultrasonic water level sensors, GPS modules, and ESP32 controllers**. Data would be transmitted using **LoRaWAN** for long-distance communication to a cloud platform. A mobile application would display real-time alerts and warning messages to local authorities and residents. This system could help reduce disaster risks and improve emergency response during heavy rainfall and flooding.

Overall, this course has provided me with practical knowledge of IoT hardware, software, networking, cloud integration, and cybersecurity. It has strengthened my confidence in designing intelligent embedded systems and inspired me to continue exploring advanced IoT technologies for solving real-world problems.